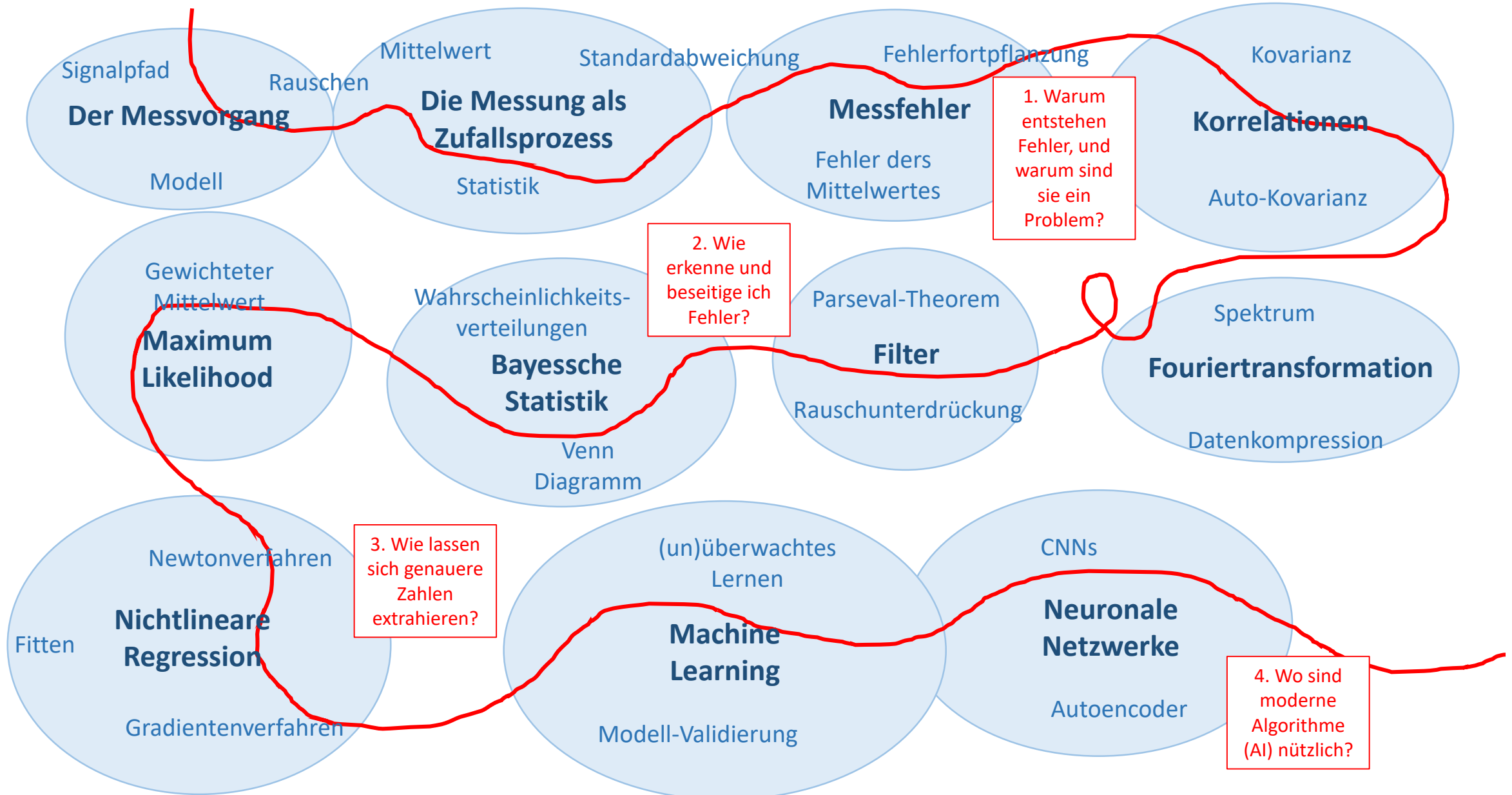


Fitten von nichtlinearen Funktionen

Der rote Faden: wie erhalte ich aussagekräftige Zahlen aus komplexen Daten?



Themen dieser Vorlesung:

- Nichtlineare schätzung der kleinsten Quadrate
- Gradientenverfahren
- Newtonverfahren
- Marquardts Methode
- Berechnen der Kovarianzmatrix aus der Hesse Matrix

Repetition

1. Was ist eine Likelihood Funktion

2. Welche Klasse von Funktionen können wir mit der linearen Regression an einen Datensatz fitten?

Nichlineare schätzung der kleinsten Quadrate

Erinnerung:

Um die lineare Funktion $a_0 - a_1 x_n$ an einen gemessenen Datensatz $y_1, y_2, \dots, y_N$ gemessen als Funktion der Unabhängigen Variable $x_1, x_2, \dots, x_N$ zu fitten haben wir die Summe der Abstandsquadrate (Residuen) minimiert:

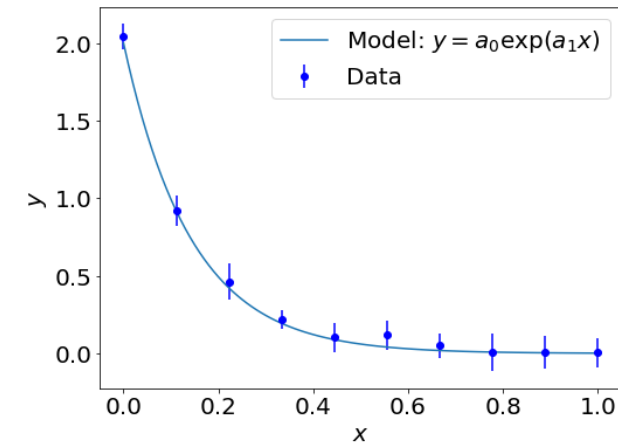
$$S = \sum_{n=1}^N w_n (y_n - a_0 - a_1 x_n)^2$$

Im Allgemeinen heisst das für eine Funktion $f(x, a_0, a_1, \dots, a_M) = f(x, \mathbf{a})$ also wir müssen

$$S = \sum_{n=1}^N w_n (y_n - f(x, \mathbf{a}))^2$$

minimieren.

Beispiel:



(Model Parameter: $a_0 = 1$, $a_1 = 5$)

Unsere Modellfunktion ist:

$$y = a_0 \exp(a_1 x) = f(x, a_0, a_1)$$

Nichlineare schätzung der kleinsten Quadrate

Erinnerung:

Um die lineare Funktion $a_0 - a_1 x_n$ an einen gemessenen Datensatz $y_1, y_2, \dots, y_N$ gemessen als Funktion der Unabhängigen Variable $x_1, x_2, \dots, x_N$ zu fitten haben wir die Summe der Abstandsquadrate (Residuen) minimiert:

$$S = \sum_{n=1}^N w_n (y_n - a_0 - a_1 x_n)^2$$

Im Allgemeinen heisst das für eine Funktion $f(x, a_0, a_1, \dots, a_M) = f(x, \mathbf{a})$ also wir müssen

$$S = \sum_{n=1}^N w_n (y_n - f(x, \mathbf{a}))^2$$

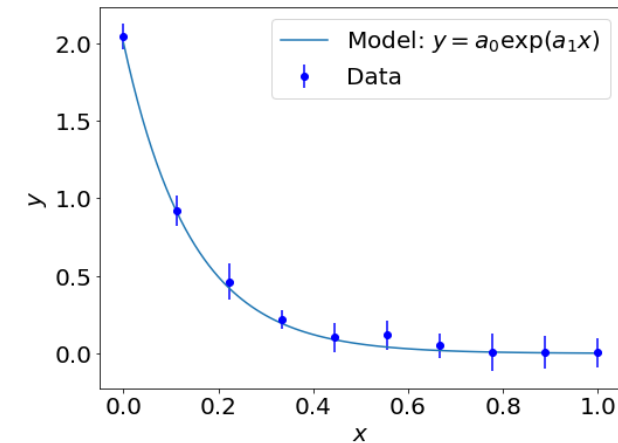
minimieren.

Wir können nun S als Funktion von „allen“ $\mathbf{a}$ berechnen.

Ziel: **Finde das Minimum von S**

Problem: **Im Allgemeinen keine geschlossene Lösung – Iterative Ansatz notwendig**

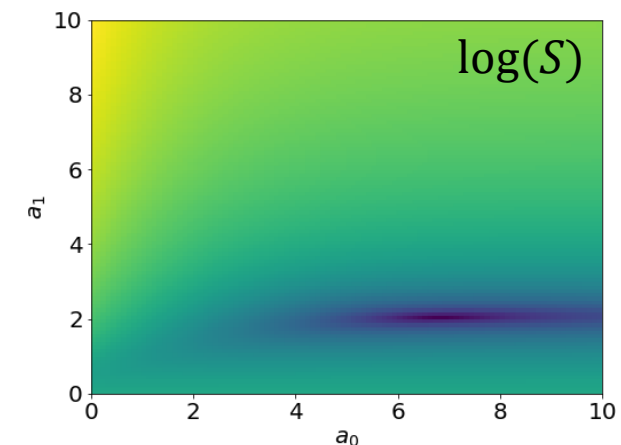
Beispiel:



(Model Parameter: $a_0 = 1$, $a_1 = 5$)

Unsere Modellfunktion ist:

$$y = a_0 \exp(a_1 x) = f(x, a_0, a_1)$$



1. Gradientenverfahren

Wir starten mit einer initialen Schätzung der Werte von $\mathbf{a}$: $\mathbf{a}_g$ und bewegen uns auf der Fläche S entlang des steilsten Gradienten den wir aus der Linearisierung (Taylor Entwicklung erster Ordnung) erhalten. Dieser Methode folgen wir iterativ zum Minimum von S :

$$S(\mathbf{a}) \approx S(\mathbf{a}_g) + \sum_{m=0}^M \left. \frac{\partial S}{\partial a_m} \right|_{\mathbf{a}_g} \Delta a_m$$

Mit $\Delta a_m = a_m - a_{gm}$

In Vektorschreibweise ist das:

$$S \approx S(\mathbf{a}_g) + \Delta \mathbf{a}^T \nabla S$$

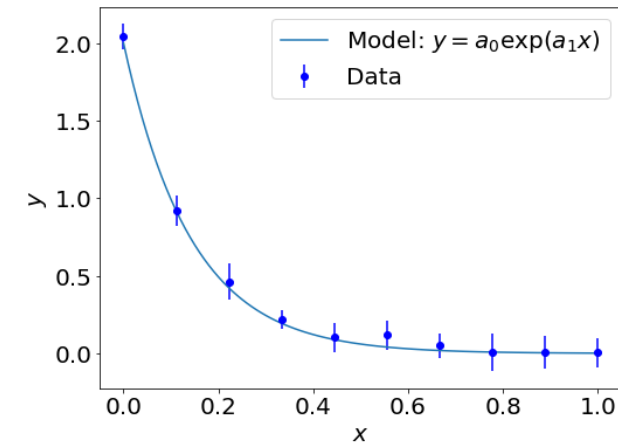
Hier ist $\nabla S = \begin{pmatrix} \frac{\partial S}{\partial a_0} \\ \vdots \\ \frac{\partial S}{\partial a_M} \end{pmatrix}$ der Gradient von S .

Damit ist die Korrektur von $\mathbf{a}_g$ Richtung Minimum gegeben durch:

$$\Delta \hat{a}_m = -\alpha \left. \frac{\partial S}{\partial a_m} \right|_{\mathbf{a}_g}$$

Wobei der Parameter α die „Grösse“ der Korrektur skaliert.

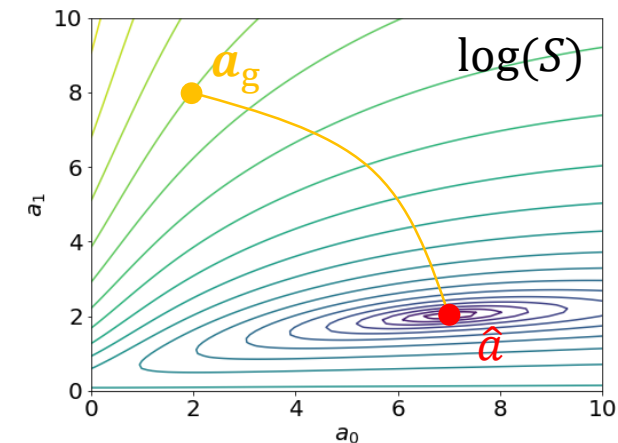
Beispiel:



(Model Parameter: $a_0 = 1$, $a_1 = 5$)

Unsere Modellfunktion ist:

$$y = a_0 \exp(a_1 x) = f(x, a_0, a_1)$$



1. Gradientenverfahren

Die Korrektur von a_g Richtung Minimum ist gegeben durch:

$$\Delta \hat{a}_m = -\alpha \left. \frac{\partial S}{\partial a_m} \right|_{a_g}$$

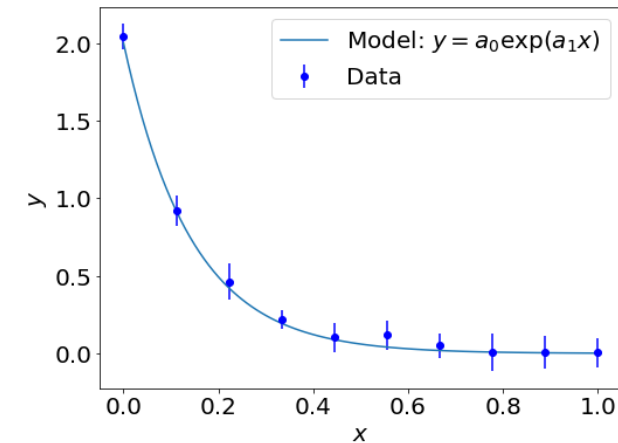
Damit sind die neuen Werte für die Fitparameter:

$$a_m = a_{gm} + \Delta \hat{a}_m$$

Diese Prozedur wird wiederholt bis die Fitfunktion, die Daten hinreichend gut beschreibt.

Frage: Wie können wir das entscheiden?

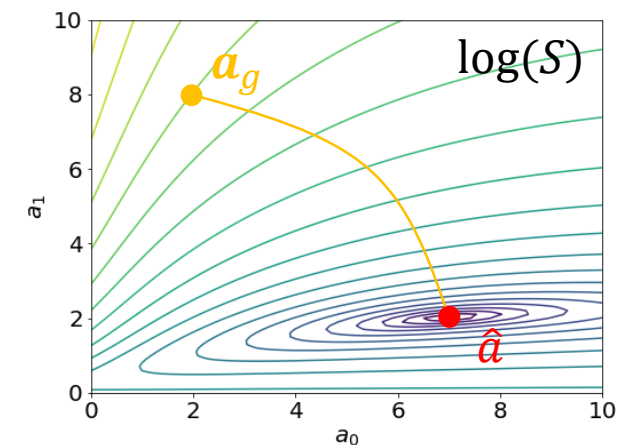
Beispiel:



(Model Parameter: $a_0 = 1$, $a_1 = 5$)

Unsere Modellfunktion ist:

$$y = a_0 \exp(a_1 x) = f(x, a_0, a_1)$$



1. Gradientenverfahren

Die Korrektur von $\mathbf{a}_g$ Richtung Minimum ist gegeben durch:

$$\Delta \hat{a}_m = -\alpha \left. \frac{\partial S}{\partial a_m} \right|_{\mathbf{a}_g}$$

Damit sind die neuen Werte für die Fitparameter:

$$a_m = a_{gm} + \Delta \hat{a}_m$$

Diese Prozedur wird wiederholt bis die Fitfunktion, die Daten hinreichend gut beschreibt.

Frage: Wie können wir das entscheiden?

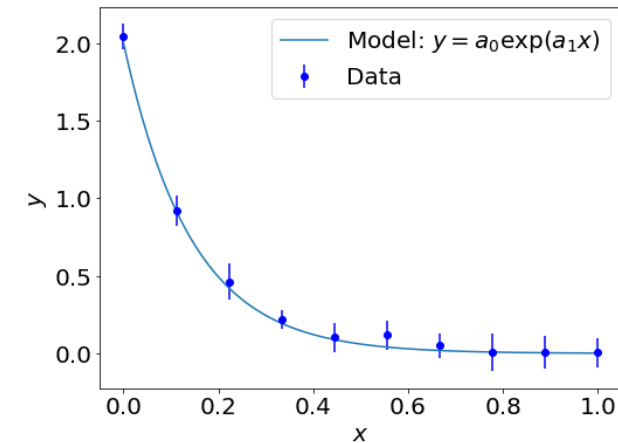
Wir können z.B. abbrechen wenn der Gradient sehr klein wird $\nabla S < \epsilon$. Das tatsächliche Minimum ist erreicht wenn $\nabla S = 0$

Die initialen Schätzwerte $\mathbf{a}_g$ sowie der Parameter α müssen möglicherweise angepasst werden um die Konvergenz des Algorithmus zu erreichen.

Siehe Beispiel Python Notebook:

„Non-Linear Fitting - Gradient Descent - single Variable“

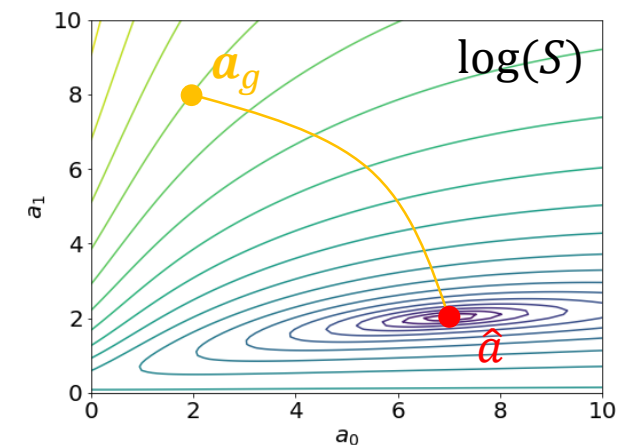
Beispiel:



(Model Parameter: $a_0 = 1$, $a_1 = 5$)

Unsere Modellfunktion ist:

$$y = a_0 \exp(a_1 x) = f(x, a_0, a_1)$$



1. Gradientenverfahren

Die Korrektur von $\mathbf{a}_g$ Richtung Minimum ist gegeben durch:

$$\Delta \hat{a}_m = -\alpha \left. \frac{\partial S}{\partial a_m} \right|_{\mathbf{a}_g}$$

Damit sind die neuen Werte für die Fitparameter:

$$a_m = a_{gm} + \Delta \hat{a}_m$$

Diese Prozedur wird wiederholt bis die Fitfunktion, die Daten hinreichend gut beschreibt.

Frage: Wie können wir das entscheiden?

Wir können z.B. abbrechen wenn der Gradient sehr klein wird $\nabla S < \epsilon$. Das tatsächliche Minimum ist erreicht wenn $\nabla S = 0$

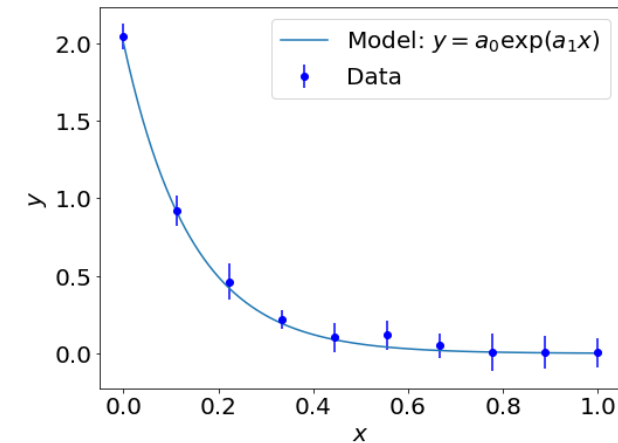
Die initialen Schätzwerte $\mathbf{a}_g$ sowie der Parameter α müssen möglicherweise angepasst werden um die Konvergenz des Algorithmus zu erreichen.

Je näher wir dem Optimum kommen, desto kleiner werden die Schritte.

Lösung:

Anpassen von α bei jedem Schritt.

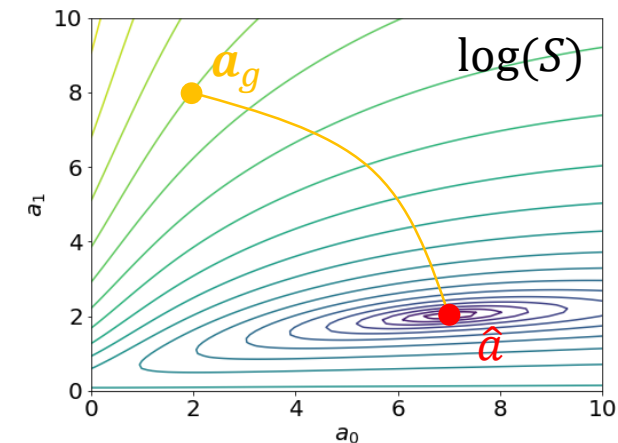
Beispiel:



(Model Parameter: $a_0 = 1$, $a_1 = 5$)

Unsere Modellfunktion ist:

$$y = a_0 \exp(a_1 x) = f(x, a_0, a_1)$$



2. Newtonverfahren

Anstelle der einfachen Linearisierung von S um $\mathbf{a}_g$ (Taylor Entwicklung erste Ordnung), können wir auch den zweiten Term der Taylor Entwicklung berücksichtigen:

$$S \approx S(\mathbf{a}_g) + \sum_{m=0}^M \left. \frac{\partial S}{\partial a_m} \right|_{\mathbf{a}_g} \Delta a_m + \frac{1}{2} \sum_{m=0}^M \sum_{k=0}^M \left. \frac{\partial^2 S}{\partial a_m \partial a_k} \right|_{\mathbf{a}_g} \Delta a_m \Delta a_k = S'(\mathbf{a})$$

Wir approximieren S also durch einen Paraboloid. Der nächste Iterationsschritt ist gegeben durch das Minimum von S' .

Das bedeutet das für alle Δa_l : $\frac{\partial S'}{\partial \Delta a_l} = 0$:

$$\left. \frac{\partial S}{\partial a_l} \right|_{\mathbf{a}_g} + \sum_{m=0}^M \left. \frac{\partial^2 S}{\partial a_l \partial a_m} \right|_{\mathbf{a}_g} \Delta \hat{a}_m = 0$$

Hier ist $\Delta \hat{\mathbf{a}}$ der Vektor mit den Korrekturen für die initialen Parameter.

Dies können wir in Matrixschreibweise schreiben:

$$\begin{pmatrix} \frac{\partial^2 S}{\partial a_0^2} & \cdots & \frac{\partial^2 S}{\partial a_0 \partial a_M} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 S}{\partial a_M \partial a_0} & \cdots & \frac{\partial^2 S}{\partial a_M^2} \end{pmatrix} \begin{pmatrix} \Delta \hat{a}_0 \\ \vdots \\ \Delta \hat{a}_M \end{pmatrix} = - \begin{pmatrix} \frac{\partial S}{\partial a_0} \\ \vdots \\ \frac{\partial S}{\partial a_M} \end{pmatrix}$$

2. Newtonverfahren

Anstelle der einfachen Linearisierung von S um $\mathbf{a}_g$ (Taylor Entwicklung erste Ordnung), können wir auch den zweiten Term der Taylor Entwicklung berücksichtigen:

$$S \approx S(\mathbf{a}_g) + \sum_{m=0}^M \left. \frac{\partial S}{\partial a_m} \right|_{\mathbf{a}_g} \Delta a_m + \frac{1}{2} \sum_{m=0}^M \sum_{k=0}^M \left. \frac{\partial^2 S}{\partial a_m \partial a_k} \right|_{\mathbf{a}_g} \Delta a_m \Delta a_k = S'(\mathbf{a})$$

Dies können wir in Matrixschreibweise schreiben:

$$\underbrace{\begin{pmatrix} \frac{\partial^2 S}{\partial a_0^2} & \cdots & \frac{\partial^2 S}{\partial a_0 \partial a_M} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 S}{\partial a_M \partial a_0} & \cdots & \frac{\partial^2 S}{\partial a_M^2} \end{pmatrix}}_{\text{Hesse Matrix } \mathbf{H}} \underbrace{\begin{pmatrix} \Delta \hat{a}_0 \\ \vdots \\ \Delta \hat{a}_M \end{pmatrix}}_{\text{Gradient } \nabla S} = - \begin{pmatrix} \frac{\partial S}{\partial a_0} \\ \vdots \\ \frac{\partial S}{\partial a_M} \end{pmatrix}$$

Und auch der Ausdruck für S' :

$$S' = S(\mathbf{a}_g) + \Delta \mathbf{a}^T \nabla S + \frac{1}{2} \Delta \mathbf{a}^T \mathbf{H} \Delta \mathbf{a}$$

2. Newtonverfahren

Das Gleichungssystem für die Nullstellenfindung von S in Matrixschreibweise ist:
(Wobei alle Ableitungen bei $\mathbf{a}_g$ ausgewertet werden)

$$\underbrace{\begin{pmatrix} \frac{\partial^2 S}{\partial a_0^2} & \cdots & \frac{\partial^2 S}{\partial a_0 \partial a_M} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 S}{\partial a_M \partial a_0} & \cdots & \frac{\partial^2 S}{\partial a_M^2} \end{pmatrix}}_{\text{Hesse Matrix } \mathbf{H}} \underbrace{\begin{pmatrix} \Delta \hat{a}_0 \\ \vdots \\ \Delta \hat{a}_M \end{pmatrix}}_{\text{Gradient } \nabla S} = - \begin{pmatrix} \frac{\partial S}{\partial a_0} \\ \vdots \\ \frac{\partial S}{\partial a_M} \end{pmatrix}$$

Also:

$$\mathbf{H} \Delta \hat{\mathbf{a}} = -\nabla S$$

Und damit erhalten wir für die Korrekturen zu den initialen Parametern:

$$\Delta \hat{\mathbf{a}} = -\mathbf{H}^{-1} \nabla S$$

Die neuen Parameter sind dann: $\hat{\mathbf{a}} = \mathbf{a}_g + \Delta \hat{\mathbf{a}}$

Auch hier fügen wir für bessere Konvergenz einen Skalierungsfaktor ($\alpha < 1$) ein:

$$\hat{\mathbf{a}} = \mathbf{a}_g + \alpha \Delta \hat{\mathbf{a}}$$

Siehe Beispiel Python Notebook:
„Non-Linear Fitting - Gradient Decent -
single Variable“

Vergleich der beiden Methoden

Wir berechnen die Korrektur als:

- Gradientenverfahren:

$$\Delta \hat{\mathbf{a}} = -\alpha \nabla S$$

- Newtonverfahren:

$$\Delta \hat{\mathbf{a}} = -\alpha \mathbf{H}^{-1} \nabla S$$

Eine intuitive und einfache Verbesserung der Methode des Gradientenverfahren ist also α dynamisch anzupassen, in dem wir es mit der inversen Krümmung von S skalieren:

$$\Delta \hat{a}_m = -\alpha \left(\frac{\partial^2 S}{\partial a_m^2} \right)^{-1} \frac{\partial S}{\partial a_m}$$

Grundsätzlich ist es wünschenswert die Vorteile von beiden Methoden zu kombinieren und ihre Nachteile zu kompensieren.

3. Marquardts Methode

Wir wollen die Verfahren 1 und 2 kombinieren um schnelle Konvergenz in allen Regimen zu erreichen.

- Wenn wir weit weg sind vom Optimum (wenn die 2te Ableitung von S gross ist) wollen wir ein konstantes α benutzen.
- Wenn wir dem Optimum näher kommen (wenn die 2te Ableitung von S klein wird) möchten wir die Korrektur mit der inversen Hesse Matrix skalieren.

Eine Methode das zu erreichen wurde von D. Marquardt vorgeschlagen, in dem die Matrix $\mathbf{H}$ ersetzt wird durch:

$$\mathbf{H}'_{i,j} = \begin{cases} \mathbf{H}_{ij}(1 + \alpha) & i = j \\ \mathbf{H}_{ij} & i \neq j \end{cases}$$

Wir benutzen $\mathbf{H}'$ um $\hat{\mathbf{a}}$ zu berechnen:

$$\Delta \hat{\mathbf{a}} = - (\mathbf{H}')^{-1} \nabla S$$

Falls α sehr gross wird vereinfacht sich dies zu:

$$\Delta \hat{a}_m = \frac{1}{(1 + \alpha)} \left(\frac{\partial^2 S}{\partial a_m^2} \right)^{-1} \frac{\partial S}{\partial a_m}$$

Das ist sehr ähnlich wie unsere einfache Verbesserung des Gradientenverfahrens.

Unsicherheit der Fitparameter

Mit diesen Methoden erhalten wir nicht automatisch die Kovarianzmatrix aus der Normalform, wie bei der linearen Minimierung der Summer der Abstandsqadrate.

Aber wir können die Kovarianzmatrix aus der Hesse Matrix berechnen:

Hierzu nehmen wir an, dass wir die Werte $\hat{\mathbf{a}}$, die S minimieren, mit einer der vorher beschriebenen Methoden bestimmt haben:

$$S_{\min} = \sum_{n=0}^N w_n (y_n - f(x_n, \hat{\mathbf{a}}))^2$$

Wir linearisieren f um $\hat{\mathbf{a}}$ mittels einer Taylor Entwicklung erster Ordnung:

$$f(x, \mathbf{a}) \approx f(x, \hat{\mathbf{a}}) + \sum_{k=0}^M \left. \frac{\partial f(x, \mathbf{a})}{\partial a_k} \right|_{\hat{\mathbf{a}}} (a_k - \hat{a}_k) = f(x, \hat{\mathbf{a}}) + \sum_{k=0}^M \frac{\partial f(x, \mathbf{a})}{\partial a_k} \Delta a_k$$

Mit $\Delta a_m = a_m - \hat{a}_m$ und von nun an werden alle Ableitungen an der Position $\hat{\mathbf{a}}$ ausgewertet.

Hiermit können wir S abschätzen als:

$$S \approx \sum_{n=0}^N w_n \left(y_n - f(x_n, \hat{\mathbf{a}}) - \sum_{m=0}^M \frac{\partial f(x_n, \mathbf{a})}{\partial a_m} \Delta a_m \right)^2$$

Unsicherheit der Fitparameter

Durch Linearisierung von f um $\hat{\mathbf{a}}$ können wir S abschätzen als:

$$\begin{aligned}
 S &\approx \sum_{n=0}^N w_n \left(y_n - f(x_n, \hat{\mathbf{a}}) - \sum_{m=0}^M \frac{\partial f(x_n, \mathbf{a})}{\partial a_m} \Delta a_m \right)^2 = \\
 &= \underbrace{\sum_{n=0}^N w_n (y_n - f(x_n, \hat{\mathbf{a}}))^2}_{= S_{\min}} - 2 \underbrace{\sum_{n=0}^N \left(w_n (y_n - f(x_n, \hat{\mathbf{a}})) \sum_{m=0}^M \frac{\partial f(x_n, \mathbf{a})}{\partial a_m} \Delta a_m \right)}_{= \frac{\partial S}{\partial \Delta a_m} \Big|_{\Delta a_m=0} = 0} + \sum_{n=0}^N w_n \underbrace{\left(\sum_{m=0}^M \frac{\partial f(x_n, \mathbf{a})}{\partial a_m} \Delta a_m \right)^2}_{= \sum_{m=0}^M \frac{\partial f(x_n, \mathbf{a})}{\partial a_m} \Delta a_j \sum_{k=0}^M \frac{\partial f(x_n, \mathbf{a})}{\partial a_k} \Delta a_k} = \\
 &= S_{\min} + \sum_{m=0}^M \sum_{k=0}^M \Delta a_m \Delta a_k \sum_{n=0}^N w_n \frac{\partial f(x_n, \mathbf{a})}{\partial a_m} \frac{\partial f(x_n, \mathbf{a})}{\partial a_k} = \\
 &= S_{\min} + \Delta \mathbf{a}^T \mathbf{N} \Delta \mathbf{a}
 \end{aligned}$$

Unsicherheit der Fitparameter

Durch Linearisierung von f um $\hat{\mathbf{a}}$ können wir S abschätzen als:

$$S = S_{\min} + \Delta \mathbf{a}^T \mathbf{N} \Delta \mathbf{a}$$

So haben wir das nichtlineare Problem auf ein lineares reduziert und können die Methoden anwenden, die wir für den linearen Fall etabliert haben. Somit können wir im Prinzip die Kovarianzmatrix $\mathbf{C} = \mathbf{N}^{-1}$ berechnen.

Um die explizite Berechnung von $\mathbf{N}$ zu vermeiden können wir uns erinnern, dass:

$$S = S(\mathbf{a}_g) + \Delta \mathbf{a} \nabla S + \frac{1}{2} \Delta \mathbf{a}^T \mathbf{H} \Delta \mathbf{a}$$

Am Minimum von S , wo $\mathbf{a}_g = \hat{\mathbf{a}}$ und $\nabla S = 0$ wird das zu:

$$S = S_{\min} + \frac{1}{2} \Delta \mathbf{a}^T \mathbf{H} \Delta \mathbf{a}$$

Und somit $\frac{1}{2} \Delta \mathbf{a}^T \mathbf{H} \Delta \mathbf{a} = \Delta \mathbf{a}^T \mathbf{N} \Delta \mathbf{a}$, was bedeutet, dass $\mathbf{N} = \frac{1}{2} \mathbf{H}$

Wir können also die Kovarianzmatrix aus der Hesse Matrix berechnen:

$$\mathbf{C} = 2\mathbf{H}^{-1}$$

Siehe Beispiel Python Notebook:
„Non-Linear Fitting - Gradient Decent“

Anwendung der Python Methode SciPy Curve Fit

In Python wie auch in den meisten anderen Datenanalysewerkzeugen gibt es effiziente und zuverlässige Methoden zur nichtlinearen Kurvenanpassung.

In Python z.B. in der Bibliothek SciPy:

Importieren der Funktion:

```
from scipy.optimize import curve_fit
```

Definiere ein Modell das gefittet werden soll:

```
def expmodel(x, amplitude, gamma):  
    return amplitude*np.exp(-gamma*x)
```

```
popt, pcov = curve_fit(expmodel, x, y, sigma = np.sqrt(1/w), method='lm', absolute_sigma=True)
```

Ergebnisse:

popt – Optimale Fitparameter.

pcov – Kovarianzmatrix. Achtung, wenn nicht explizit absolute_sigma=True angegeben wird, wird die Kovarianzmatrix folgendermassen berechnet:

$$\mathbf{C} = \frac{\hat{S}_{\min}}{N - M - 1} \mathbf{N}^{-1}$$

Zusammenfassung

- Wir können für jede Funktion die Kostenfunktion S (die Summe der Abstandsquadrate) definieren.
- Minimieren von S erlaubt uns die Funktionsparameter zu finden für die die Funktion die Daten am besten beschreibt.
- Dazu folgen wir iterativ dem Gradienten von S zum Minimum.
- Die Unsicherheit der Fitparameter können wir mit Hilfe der Hesse-Matrix berechnen.
- Die meisten Datenanalyse Werkzeuge stellen effiziente und zuverlässige Implementation dieser Methoden zu Verfügung. In Python z.B. `scipy.curve_fit`